1. Session Overview and Analytical Logic

1.1 The Session 2 Learning Path

Session 2 extends the basic **NLP pipeline** from cleaning individual strings to finding patterns across a **corpus** . The central progression is:

1. inspect and preprocess documents;
2. measure local relationships with **n-grams** , **associations** , and **collocations** ;
3. represent documents numerically with **bag of words** or **TF-IDF** ;
4. discover latent structure with **topic modelling** or **clustering** ;
5. evaluate, interpret, and communicate the resulting structure.

Each step changes what the next method can discover. For example, stemming changes the vocabulary supplied to TF-IDF, while TF-IDF changes the geometry used by a clustering algorithm. Preprocessing is therefore part of the model design, not merely housekeeping.

1.2 Choosing an NLP Approach

The slides distinguish three broad families of approach. **Rule-based heuristics** encode human knowledge directly and are often transparent, fast, and effective when patterns are stable. **Classical machine learning** learns relationships from engineered numerical features such as TF-IDF. **Deep learning** learns richer representations from data but normally requires more data, computation, and diagnostic care.

The most complex method is not automatically the most suitable. A defensible choice follows the task, data volume, label availability, interpretability requirement, deployment constraints, and cost of errors. Session 2 concentrates on transparent exploratory methods that help the analyst understand the corpus before moving to more complex models.

> ### Deep Dive — Analysis Before Modelling
>
> An exploratory model can reveal sampling errors, duplicated documents, boilerplate, or vocabulary leakage before a predictive model is trained. This makes exploratory text analysis a form of **quality assurance** as well as a source of insight.

2. N-Grams and Lexical Relationships

2.1 N-Grams Preserve Local Order

An **n-gram** is a contiguous sequence of n items. With word tokens, a **unigram** contains one word, a **bigram** two adjacent words, and a **trigram** three. For the sequence I love learning, the bigrams are I love and love learning, while the trigram is I love learning.

N-grams recover some local order that a unigram bag-of-words representation discards. They can expose recurring entities and phrases such as **machine learning** or **social media** , but longer n-grams become sparse because exact sequences repeat less often. Their counts also depend strongly on tokenisation, casing, punctuation, stopword removal, and stemming.

```
from collections import Counter
from nltk import ngrams
def get_top_ngrams(tokens, n, top_k=10):
counts = Counter(ngrams(tokens, n))
return counts.most_common(top_k)
top_bigrams = get_top_ngrams(tokenized_words, 2)
top_trigrams = get_top_ngrams(tokenized_words, 3)
```

2.2 Syntagmatic, Paradigmatic, and Phonetic Relationships

Words can be related in different ways. A **syntagmatic relationship** links words that combine in a sequence, such as strong coffee; n-grams and collocations are designed to capture this type of relation. A **paradigmatic relationship** links words that could substitute for one another in a context, such as coffee and tea. A **phonetic relationship** is based on sound rather than semantic role.

This distinction matters because co-occurrence does not prove synonymy. Two words may frequently appear together precisely because they play different, complementary roles.

2.3 Word Association and Pointwise Mutual Information

Raw bigram frequency asks, “How often did these words occur next to each other?” **Pointwise Mutual Information (PMI)** instead asks, “How surprising is their joint occurrence relative to independence?” It compares the observed joint probability P(x,y) with the baseline P(x)P(y).

- **PMI > 0** means the words occur together more often than expected under independence.
- **PMI = 0** is consistent with the independence baseline.
- **PMI < 0** means the words occur together less often than expected.

The Part 1 notebook ranks candidate bigrams with NLTK:

```
from nltk.collocations import BigramAssocMeasures, BigramCollocationFinder
measures = BigramAssocMeasures()
finder = BigramCollocationFinder.from_words(tokenized_words)
top_associations = finder.nbest(measures.pmi, 20)
```

$$\mathrm{PMI}(x, y) = \log_2\left(\frac{P(x, y)}{P(x)P(y)}\right)$$

Association relative to independence; the log base controls the unit.

Deep Dive — Why Rare Pairs Can Dominate PMI

PMI can assign a high score to a pair observed only once when both words are individually rare. Apply a minimum-frequency filter and inspect counts alongside PMI. Alternative association statistics include the **likelihood-ratio test** and **chi-square test** , which encode different evidence about departure from independence.

2.4 Association Is Not the Same as Collocation

A **word association** is a statistical relationship, whereas a **collocation** is a conventional multiword expression whose components repeatedly occur together, such as natural language or machine learning. High association scores propose candidates; linguistic interpretation decides whether a candidate is a meaningful collocation.

The notebook demonstrates **likelihood ratio** for ranking collocations:

```
measures = BigramAssocMeasures()
finder = BigramCollocationFinder.from_words(tokenized_words)
top_collocations = finder.nbest(measures.likelihood_ratio, 20)
```

Do not compare rankings blindly. Frequency, PMI, likelihood ratio, and chi-square answer related but non-identical questions and can prioritise different word pairs.

2.5 Concordance Restores Context

A **concordance** lists the contexts in which a target term occurs. It is a bridge between corpus-level measurement and close reading: after a frequency or association result identifies a pattern, concordance lines reveal how the word is actually being used.

```
from nltk.text import Text
corpus_text = Text(tokenized_words)
corpus_text.concordance("learning")
```

Concordance inspection can reveal polysemy, quotation, negation, sarcasm, boilerplate, or named-entity uses that an aggregate count hides.

3. Preprocessing as an Analytical Decision

3.1 Stemming and Lemmatisation

Stemming applies heuristic rules to strip affixes, often producing a root-like form that is not a dictionary word. It is fast and can reduce vocabulary size, but may merge unrelated forms or leave related forms separate. **Lemmatisation** uses vocabulary and grammatical information to return a dictionary base form, or lemma. It is usually more interpretable but requires a linguistic model and may be slower.

For example, a lemmatiser can map cats to cat, running to run, and the comparative adjective better to good when the part of speech is recognised. A stemmer may instead return compressed forms intended for matching rather than reading.

3.2 Pipeline Order Changes the Result

The Part 1 notebook defines a reusable TextPreprocessor whose order is explicit:

1. add spacing after closing parentheses;
2. remove punctuation;
3. lowercase;
4. remove stopwords;
5. normalise whitespace;
6. stem tokens.

This sequence solves a corpus-specific issue in which a token ending with) could be joined incorrectly after punctuation removal. The broader lesson is that a pipeline should be tested on representative edge cases, not assumed correct because every individual function runs.

```
def preprocess(self, text):
text = self.add_space_after_parenthesis(text)
text = self.remove_punctuation(text)
text = self.to_lowercase(text)
text = self.remove_stopwords(text)
text = self.remove_extra_whitespace(text)
text = self.stem_words(text)
return text
```

3.3 Preserve Information Required by the Task

Removing punctuation may destroy emoticons, hashtags, code syntax, sentence boundaries, or abbreviation cues. Lowercasing may merge distinct named entities. Removing stopwords may delete negation. Stemming may make topic labels harder to interpret. Before deleting a feature, ask whether it carries signal for the intended analysis.

A reproducible workflow records the original text, the transformed text, every preprocessing parameter, package versions, and random seeds. It also inspects examples before and after transformation.

> **Deep Dive — Corpus-Specific Stopwords**
>
> Very frequent source names such as cnn can dominate a news corpus without answering the research question. Adding them to a custom stopword list may be justified, but the list should be documented and reviewed so that substantive terms are not removed merely because they are common.

4. From Text to Numerical Features

4.1 Bag of Words and the Document-Term Matrix

Machine-learning algorithms require numerical inputs. A **bag-of-words (BoW)** representation counts terms while largely ignoring word order. Across a corpus, these counts form a **document-term matrix (DTM)** in which rows represent documents and columns represent vocabulary terms.

The DTM is usually **sparse** because each document uses only a small fraction of the corpus vocabulary. Its advantages are simplicity, efficiency, and interpretability. Its limits include loss of order, weak handling of synonymy and polysemy, and sensitivity to vocabulary construction.

4.2 Term Frequency–Inverse Document Frequency

TF-IDF combines local importance with corpus-level rarity. **Term frequency (TF)** measures how strongly term t occurs in document d. **Inverse document frequency (IDF)** downweights terms appearing in many documents. Their product gives a large value to a term that is prominent in a particular document but not ubiquitous across the corpus.

TF-IDF improves many retrieval and clustering tasks, but it does not encode word order or meaning. Implementations may use raw or normalised TF, smoothed IDF, sublinear scaling, and vector normalisation; these choices should be reported.

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df["text"])
vocabulary = vectorizer.vocabulary_
```

$$\mathrm{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

One common normalised term-frequency definition.

$$\mathrm{IDF}(t) = \log\left(\frac{N}{\mathrm{df}(t)}\right)$$

N is corpus size; df(t) is the number of documents containing t.

$$\mathrm{TFIDF}(t, d) = \mathrm{TF}(t, d) \times \mathrm{IDF}(t)$$

Distinctive-in-document weighting; implementations may smooth or normalise.

Deep Dive — Fit Leakage

In a predictive task, fit the vectoriser only on the training data, then call transform on validation and test data. Fitting vocabulary or IDF weights on all documents leaks information from evaluation data. In a purely exploratory analysis of one fixed corpus, fitting the complete corpus may be appropriate if that scope is stated.

5. Exploratory Visualisation and Data Storytelling

5.1 Match the Visual to the Analytical Question

The slides present visualisation as part of **sensemaking** , not decoration. A **word cloud** gives a fast, approximate view of frequent terms, but area and layout make exact comparison difficult. A sorted **bar chart** supports more precise comparison. A **histogram** shows a distribution; a **boxplot** exposes spread and outliers; a **scatter plot** displays relationships between two numerical dimensions; a **network graph** can show repeated word connections.

Heatmaps, topic maps, and dimensionality-reduction plots can summarise complex models, but every visual inherits the assumptions of the underlying preprocessing and representation. Raw-data problems should be addressed before styling a chart.

5.2 Bigram Networks

The Part 1 notebook constructs an undirected **NetworkX** graph in which words are nodes, frequent bigrams are edges, and edge width or colour represents bigram frequency. This can reveal hubs and phrase communities, but a crowded graph quickly becomes unreadable. The notebook limits display to the 50 most frequent bigrams and scales widths to a visible range.

Graph structure should not be overinterpreted: an edge means adjacency in the chosen corpus and preprocessing pipeline, not necessarily causality or semantic equivalence.

5.3 Build a Defensible Data Story

A data story links evidence to a question and an audience. A useful sequence is **context → analytical question → method → evidence → interpretation → limitation → implication** . The chart should make the comparison visible; the prose should explain why it matters. For AT1, this means connecting every visual to a textual pattern and avoiding claims stronger than the corpus supports.

6. Topic Modelling with Latent Dirichlet Allocation

6.1 What Topic Modelling Discovers

Topic modelling is an unsupervised approach for discovering recurring patterns of word use. A topic is represented as a probability distribution over words, and each document is represented as a mixture of topics. Topics are latent statistical structures, not pre-labelled truths; the analyst assigns human-readable labels after inspecting high-probability words and representative documents.

6.2 Simplified LDA Intuition

Latent Dirichlet Allocation (LDA) assumes that each document has a distribution over topics and each topic has a distribution over words. In a simplified generative view, the model repeatedly chooses a topic for a word position from the document's topic mixture, then chooses a word from that topic's vocabulary distribution.

The number of topics K is selected by the analyst. Too few topics can merge distinct themes; too many can split coherent themes or create redundant, unstable topics. LDA also relies on a bag-of-words assumption, so word order is not directly modelled.

6.3 Notebook Workflow: Cleaning to Corpus

The Part 2 notebook uses the **20 Newsgroups** dataset and follows this sequence:

1. remove email addresses, newlines, quotes, and header metadata with regular expressions;
2. tokenise and de-accent using Gensim simple_preprocess;
3. detect frequent bigrams and trigrams with Phrases and Phraser;
4. remove NLTK English stopwords;
5. lemmatise only nouns, adjectives, and verbs with spaCy;
6. build a Gensim Dictionary mapping tokens to integer IDs;
7. convert documents to (token_id, token_count) BoW tuples;
8. train, inspect, save, and visualise an LDA model.

The phrase parameters are part of the model design: the notebook uses min_count=3 and threshold=100 for bigram detection. Its lemmatiser loads en_core_web_sm with parsing and named-entity recognition disabled because those components are not needed for this workflow.

6.4 Training the Gensim Model

The current notebook trains 20 topics with a fixed random state, ten passes, automatic alpha estimation, and per-word topic information:

```
from gensim.models.ldamodel import LdaModel
lda_model = LdaModel(
corpus=corpus,
id2word=dictionary,
num_topics=20,
random_state=100,
update_every=1,
chunksize=100,
passes=10,
alpha="auto",
per_word_topics=True,
)
```

The notebook also saves and reloads the fitted model. The checked-in output contains a stale NameError at the reload cell because LdaModel had not been defined in that saved execution order. Running the import/training cell first, or importing LdaModel before loading, resolves the order dependency.

6.5 Count-Based LDA and the TF-IDF Extension

The main workflow uses word counts, which align with LDA's probabilistic word-generation interpretation. The notebook then experiments with a TF-IDF-transformed corpus and LdaMulticore. This is a useful empirical comparison, but TF-IDF is not the canonical input implied by the standard LDA generative model. Compare coherence, stability, representative documents, and interpretability rather than assuming the transformed version is automatically better.

6.6 Interpreting Topics with pyLDAvis

pyLDAvis provides an interactive view of topic prevalence, separation, and salient terms. Large, well-separated topic circles suggest distinct high-level patterns; overlapping circles suggest related or insufficiently separated topics. The term panel supports interpretation but does not replace reading representative documents.

```
import pyLDAvis
from pyLDAvis.gensim import prepare
pyLDAvis.enable_notebook()
visualisation = prepare(lda_model, corpus, dictionary)
```

Deep Dive — Topic Quality Is Multi-Dimensional

A topic model should be checked for **semantic coherence** , separation, stability across seeds or samples, usefulness for the research question, and sensitivity to preprocessing. A numerically strong model can still produce topics that are hard to label or irrelevant to the decision being supported.

7. Text Clustering Foundations

7.1 Clustering Without Known Labels

Clustering groups documents so that texts within a cluster are more similar to one another than to texts in other clusters. Unlike supervised classification, the algorithm is not given target labels. The resulting cluster IDs are arbitrary; Cluster 0 does not inherently represent a particular subject or rank.

Clustering depends on three linked decisions: the text representation, the similarity or distance measure, and the clustering algorithm. Changing any one can change the result.

7.2 Similarity and Distance in Text Space

Euclidean distance measures straight-line separation and is sensitive to scale and vector magnitude. **Manhattan distance** sums coordinate-wise absolute differences. **Cosine similarity** compares vector direction, making it useful for sparse text vectors where document length should have less influence; **cosine distance** is commonly defined as one minus cosine similarity.

High-dimensional sparse spaces create a **curse of dimensionality** : distances can become less discriminative, computation grows, and visual intuition fails. Feature selection, dimensionality reduction, suitable metrics, and careful validation can help, but each changes the geometry being analysed.

$$d_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_i (x_i - y_i)^2}$$

Euclidean distance: straight-line separation in feature space.

$$d_1(\mathbf{x}, \mathbf{y}) = \sum_i |x_i - y_i|$$

Manhattan distance: coordinate-wise absolute separation.

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}$$

Cosine distance compares direction and reduces magnitude effects.

8. Hierarchical Clustering

8.1 Agglomerative and Divisive Strategies

Agglomerative hierarchical clustering starts with each document as its own cluster and repeatedly merges the closest pair. **Divisive hierarchical clustering** starts with one cluster and repeatedly splits it. The result is a hierarchy rather than only one flat partition.

A **dendrogram** visualises the merge sequence. Leaves represent observations; merge height represents the linkage distance at which groups combine. Cutting the dendrogram at a chosen height yields a flat set of clusters.

8.2 Linkage Defines Distance Between Groups

Once clusters contain multiple documents, the algorithm needs a **linkage criterion** . Single linkage uses the closest pair across clusters and may form chains. Complete linkage uses the farthest pair and tends to produce compact groups. Average linkage averages cross-cluster distances. **Ward linkage** chooses merges that minimise the increase in within-cluster variance and is conventionally paired with Euclidean geometry.

The notebook's toy example vectorises nine short documents with TF-IDF, converts the sparse matrix to a dense array, and applies Ward linkage:

```
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(stop_words="english")
tfidf_matrix = vectorizer.fit_transform(documents)
linkage_matrix = linkage(tfidf_matrix.toarray(), method="ward")
dendrogram(linkage_matrix)
```

This is suitable for a small demonstration. Converting a large sparse text matrix to dense form can exhaust memory, so production-scale workflows require methods that preserve sparsity or reduce dimensions first.

9. K-Means and Model Selection

9.1 K-Means Iterative Optimisation

K-means partitions vectors into a specified number K of clusters. It initialises centroids, assigns each observation to the nearest centroid, recomputes each centroid as the mean of its assigned points, and repeats until assignments or the objective stabilise.

The method is efficient but assumes centroid-shaped groups in the chosen geometry. It is sensitive to initialisation, outliers, feature scale, and the chosen K. **K-means++** improves initial centroid selection, and repeated initialisations reduce the chance of reporting a poor local solution.

9.2 Elbow and Silhouette Methods

The **elbow method** plots within-cluster sum of squared distances, or inertia, across candidate values of K. Inertia always decreases as more clusters are added; the goal is to identify where additional clusters bring diminishing improvement. The elbow can be ambiguous.

The **silhouette coefficient** compares each observation's average distance to its own cluster, a(i), with its smallest average distance to another cluster, b(i). Values approach 1 for well-separated observations, sit near 0 at boundaries, and become negative when an observation may fit another cluster better.

The Part 3 notebook evaluates (K=2) through (9) with both methods, then chooses four clusters:

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
scores = {}
for k in range(2, 10):
model = KMeans(n_clusters=k, random_state=42)
labels = model.fit_predict(X)
scores[k] = silhouette_score(X, labels)
```

Selection should combine these diagnostics with cluster size, stability, representative documents, top terms, and usefulness. A peak silhouette score does not guarantee semantically meaningful clusters.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

Silhouette ranges from -1 to 1; larger values indicate clearer separation.

9.3 Interpreting and Visualising Clusters

After fitting K-means, inspect documents and distinctive terms in every cluster. When known labels exist, a **cross-tabulation** can reveal correspondence, but unsupervised clusters are not expected to reproduce an external taxonomy exactly.

The notebook uses **UMAP** to project the TF-IDF vectors into two dimensions and colours points with the **viridis** palette. UMAP coordinates are a visual aid, not the space in which the original K-means model was trained. Apparent separation can change with UMAP parameters and random seed.

```
from umap import UMAP
reducer = UMAP(n_components=2, random_state=42)
X_umap = reducer.fit_transform(X)
plt.scatter(X_umap[:, 0], X_umap[:, 1], c=labels, cmap="viridis")
```

Deep Dive — Validate the Entire Pipeline

Clustering validation should be repeated across preprocessing choices, vectoriser settings, candidate K, and random seeds. If the narrative changes radically under small reasonable changes, the apparent structure is unstable and should be reported cautiously.

10. Notebook Guide and Reproducible Setup

10.1 Part 1 — Text Analysis

Part 1 reads a 2021 CNN article dataset, inspects the dataframe, creates a word cloud, computes n-grams, ranks word associations and collocations, performs concordance analysis, executes a custom preprocessing pipeline, and visualises cleaned frequencies with a bar chart and bigram network.

Important dependencies include **pandas** , **NLTK** , **Matplotlib** , **WordCloud** , and **NetworkX** . The notebook downloads NLTK punkt_tab and stopwords. Compare results before and after preprocessing; the difference is evidence about the transformation, not simply a cosmetic improvement.

10.2 Part 2 — Topic Modelling

Part 2 uses **scikit-learn** to obtain 20 Newsgroups, then uses **Gensim** , **spaCy** , and **pyLDAvis** for preprocessing, LDA, and interpretation. Install the en_core_web_sm model in the same Python environment as the notebook kernel. A blank spaCy English pipeline can tokenise but cannot provide the POS tags and lemmas required by this notebook's POS-filtered lemmatisation.

10.3 Part 3 — Text Clustering

Part 3 begins with a small hierarchical-clustering example, then cleans and stems 20 Newsgroups documents, constructs TF-IDF features, evaluates K with elbow and silhouette plots, fits K-means, inspects clustered documents, and visualises a UMAP projection. **SciPy** , **scikit-learn** , **NLTK** , **WordCloud** , and **umap-learn** are required in addition to the common analysis stack.

10.4 Run Order and Evidence

Restart the kernel and run all cells from top to bottom before relying on notebook output. Record package versions and seeds, retain output needed to support conclusions, and distinguish a teaching demonstration from a validated analytical result. A notebook that executes is necessary but not sufficient: its conclusions must still be checked against the corpus and research question.

11. Homework and AT1 Connection

11.1 Take-Home Exercises

The homework notebook asks you to:

1. calculate word associations on a large dataset using multiple methods such as PMI and chi-square;
2. compare lemmatisation with stemming;
3. add another preprocessing step, such as removing digits, and evaluate its effect;
4. try scikit-learn topic-modelling approaches including **LSA** , **LDA** , and **NMF** ;
5. apply text clustering to another dataset and optimise the number of clusters.

For each exercise, write a short rationale before the code and a conclusion after the output. A useful conclusion states what changed, why the change matters, and what limitation remains.

11.2 Evidence for AT1

These Session 2 methods can support AT1 exploratory analysis. A strong workflow preserves an auditable preprocessing pipeline, compares alternative representations or parameters, interprets representative documents, uses quantitative diagnostics without treating them as truth, and connects every figure to the analytical question.

Avoid choosing a model solely because it produces an attractive visual. Report how the corpus was constructed, what was removed, how parameters were selected, and how uncertainty or instability affects the conclusion.

12. Learning Objectives and Revision Check

12.1 Learning Objectives

After this session, you should be able to:

1. generate and interpret unigrams, bigrams, and trigrams;
2. distinguish raw co-occurrence, statistical association, and linguistic collocation;
3. explain PMI and identify its sensitivity to rare events;
4. compare stemming and lemmatisation and justify preprocessing order;
5. construct and interpret BoW, DTM, and TF-IDF representations;
6. select visualisations that support accurate text-data comparisons;
7. explain the document-topic and topic-word structure of LDA;
8. build and interpret hierarchical and K-means text-clustering workflows;
9. use elbow and silhouette evidence to investigate, not mechanically dictate, K;
10. evaluate topic or cluster outputs using both diagnostics and close reading;
11. connect modelling choices to reproducibility, limitations, and AT1 evidence.

12.2 Revision Questions

1. Why can a rare bigram receive a high PMI score?
2. Which preprocessing choices would be risky for sentiment analysis, and why?
3. Why does TF-IDF change clustering results compared with raw counts?
4. What do an LDA topic and a document-topic mixture represent?
5. Why might count-based LDA be easier to justify than TF-IDF-based LDA?
6. How does linkage choice change a hierarchical clustering result?
7. What information do the elbow and silhouette methods provide?
8. Why should UMAP separation not be treated as direct proof of cluster quality?
9. Which documents would you inspect before naming a topic or cluster?
10. What evidence would show that an exploratory result is stable enough to report?